What changes on the dashboard

For every option in Terminal 4, and what the graph peaks mean

> **How to use this PDF**
> Terminal 4 is where you press a key to start an attack. The dashboard (Terminal 3, `http://127.0.0.1:8080`) reacts. This PDF lists, for each key, what you will see change and where. Times are typical values worked out from the project code and the attack programs. Exact numbers (especially packets per second) depend on how fast your laptop is.

The same five steps happen for every attack



The dashboard asks for new data every 1.5 seconds, so every change appears with up to 1.5 s delay. A short event can appear in only one reading.

Where to look on the dashboard

Part	What changes during an attack
Red banner (top)	Appears when something is blocked. Names the attacker and the rule.
Stat strip	alerts goes up, blocked goes up, network packets/s and throughput jump on floods, live connections changes, dropped by IPS starts counting.
Traffic graph	The attacker's line spikes. Normal hosts stay low.
Hosts table	The attacker's sent/s, pkts sent, conns and state change. The state becomes BLOCKED.
Live connections	New rows from the attacker (green ALLOWED), then a red BLOCKED row at the top.
Traffic by service	The attacked port (5000 app) gets a longer bar on floods.
Active blocks	One card per block, with the rule, the type of drop and a ttl countdown.
Activity feed	OPEN lines, then ALERT and BLOCK. About 30 s later, RECOVER.
Flow table (bottom)	A new red DROP row (priority 100 or 200).

Your graph: what are these peaks?

The graph you sent shows **normal traffic, not an attack**. You can tell from four facts:

- The vertical axis only goes up to about 50 packets per second. The attack peaks in this project are in the thousands.
- All 13 lines are small and move around. h1 and h10 (the attackers) are at 0.
- There is no red banner and nothing is BLOCKED.
- The legend values (h2 9, h3 13, h6 4 ...) are tiny.

Why do the normal lines go up and down?

The normal clients (h3 to h9) run `apps/background_traffic.py`. It sends a request to a random server at a random time, every 0.3 to 2.5 seconds. About 7 out of 100 times it sends a quick burst of 3 to 5 requests in a row. Each peak in your graph is one of these bursts, or several clients sending at the same moment. They have no pattern and they are meant to look like an office.

Line	Typical height	Why
h3	10 to 20 pps, almost flat	h3 is the heartbeat client. It sends about 2 requests per second all the time.
h2	5 to 15 pps	h2 is the main server. It answers requests from every client, so it sends more than most.
h4 to h9	0 to 15 pps, spiky	Workstations with random requests and occasional bursts.
srv2, srv3, srv4	0 to 8 pps	Servers that only answer when asked.
h1, h10	0	Attackers. They send nothing until you press a key in Terminal 4.

What an attack peak looks like, and when it is at its maximum

An attack peak is much bigger than these (thousands or more) and it is one line, in the attacker's colour. It reaches its maximum in the **first one or two readings after the attack starts**, because that is the time between the flood starting and the switch dropping the packets. Then it falls straight back to 0.

Detection path	How long the peak lasts
Packet-In rules (options 1 to 4)	Almost no time. The block is installed in milliseconds, so the peak is small or not visible. Use the feed, banner and hosts table to see these.
Polling rules (option 6, and option 5)	Up to 3 seconds. The controller only reads the counters every 3 seconds, so the flood runs until the next reading. This is why option 6 gives the big peak.

> **Three tips for reading the graph**
> 1) The vertical axis rescales by itself, so after a flood the normal lines look flat. Click **log** to see both. 2) The orange dashed line is the packet_rate limit (3000). It only shows when the axis is big enough to reach it. 3) The graph remembers the last 90 readings (about 2 minutes). Reloading the page clears it.

The comparison table

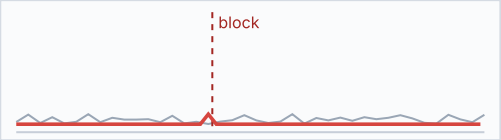
Key	Attack	Rule	Block	Detected	Graph spike	Clearest sign on the panel
1	Illegal flags	invalid_flags	IP, prio 100	0.2 s	none	banner + feed
2	Port scan	port_scan	IP, prio 100	0.8 s	tiny blip	odd-port rows in Live connections
3	Multi-IP spoof	multi_ip_spoof	Port, prio 200	0.2 s	tiny blip	banner shows "port 1 quarantined"
4	SYN flood	conn_flood	IP, prio 100	0.3 s	short blip	5000 app bar, conns up to 20
5	Slow-and-low	concurrent_flows	IP, prio 100	8 to 11 s	**none**	conns climbs to 40
6	Volumetric	packet_rate or port_flood	IP 100 or port 200	ms to 3 s	**big peak**	sent/s, graph, network packets/s
c	Coordinated	conn_flood + (packet_rate or port_flood)	two blocks	0.3 s / ms to 3 s	small + big	blocked = 2, two red cards

Option by option

Each card shows the expected graph shape (grey = normal hosts, red = attacker, dashed red = block installed) and then what changes in each part of the dashboard.

1 Illegal TCP flags

Program sends	6 packets, 0.08 s apart, using impossible flag combinations (SYN+FIN, NULL, XMAS).
Rule that fires	`invalid_flags` at 3 packets in 10 s.
Detected after	about 0.2 s (on the 3rd packet, Packet-In path)
Block installed	IP drop, priority 100, for 10.0.0.1 (h1).



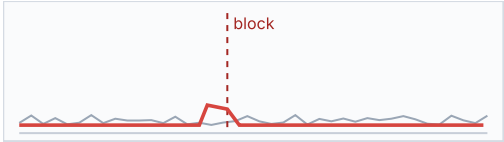
Panel part	While it runs	After the block
Graph	No visible spike. Only 6 packets are sent.	h1 line stays flat.
Red banner	Appears within one refresh (1.5 s): `h1 10.0.0.1 → invalid_flags`.	Stays until the block is released.
Hosts table, h1	**pkts sent** goes up by about 6. State changes to **BLOCKED**.	**dropped** shows a small number (0 to 3).
Live connections	A red BLOCKED row for h1 appears at the top. No ALLOWED rows for h1 (illegal packets get no fast-path rule).	Same.
Active blocks	h1, rule invalid_flags, "prio-100 ipv4_src drop", detail "3 illegal-flag packets ... in 10s", idle ttl about 30 s.	ttl counts down; entry disappears at release.
Activity feed	**ALERT** and then **BLOCK** (with response time in ms).	**RECOVER** about 30 s after the last packet.
Flow table (bottom)	A new red row: priority 100, DROP, `ip h1 → *`.	Row removed at release.

Terminal 2 shows: `[IDS ALERT] 10.0.0.1 | rule=invalid_flags ...` then `[IDS RESPONSE] 10.0.0.1 MITIGATED`.

What to say: "Packets with flag combinations that normal software never sends are detected on the third one, and the source is blocked in milliseconds."

2 Port scan

Program sends	Connection attempts to ports 1 to 40 on h2, about 10 per second (0.1 s apart).
Rule that fires	port_scan at 8 distinct destination ports in 10 s.
Detected after	about 0.8 s (on the 8th probe)
Block installed	IP drop, priority 100, for 10.0.0.1.



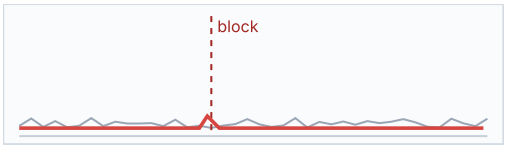
Panel part	While it runs	After the block
Graph	A very small blip (about 10 to 30 packets/s). Easy to miss on the linear scale.	Flat again.
Live connections	For a moment, several rows `h1 → h2:1`, `:2`, `:3` ... appear (service names are blank because these are not real services). The first 7 probes were allowed before the limit was reached.	They disappear about 10 s after the last packet (10 s idle timeout). The BLOCKED row for h1 stays.
Traffic by service	Several short bars for odd ports (1, 2, 3 ...).	Bars fade as the rules expire.
Hosts table, h1	**conns** briefly 5 to 8, then state **BLOCKED**.	**conns** back to 0.
Banner, Active blocks	`h1 10.0.0.1 → port_scan`; detail "8 distinct destination ports in 10s (limit 8)".	Gone at release.
Activity feed	**OPEN** lines for the probed ports, then **ALERT**, **BLOCK**.	**CLOSE** lines after about 10 s, then **RECOVER**.

Terminal 2 shows: `[IDS ALERT] 10.0.0.1 | rule=port_scan | 8 distinct destination ports ...`

What to say: "One host asked about many different ports in a few seconds. That is a scan, so it is blocked after the 8th port."

3 Multi-IP spoof

Program sends	25 SYN packets, 0.08 s apart. Each one claims a different random source IP (10.0.0.10 to 10.0.0.200) but all leave through h1's port.
Rule that fires	`multi_ip_spoof`: the same MAC sends more than 2 different IPs in 4 s.
Detected after	about 0.2 s (on the 3rd packet)
Block installed	**Port quarantine**, priority 200, on h1's switch port (port 1), plus a priority-100 drop on h1's MAC.



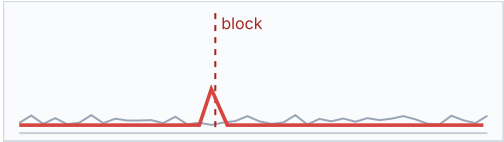
Panel part	While it runs	After the block
Graph	Tiny blip (25 packets in 2 s).	Flat.
Red banner	`h1 [src 10.0.0.x] → multi_ip_spoof (port 1 quarantined)`. The "src" is one of the fake addresses. The panel names the real host (h1) from the port number.	Gone at release.
Hosts table, h1	State **BLOCKED**, because the whole port is cut off. **dropped** counts every packet h1 sends afterwards.	Back to normal at release.
Live connections	Red BLOCKED rows: one for h1 and one for the fake source address (the fake one shows 0 dropped).	Same.
Active blocks	Shows `prio-200 in_port=1 drop` and the detail "MAC ... rotated N source IPs in 4.0s".	ttl counts down.
Flow table (bottom)	Two red rows: priority 200 `in_port=1 (h1)` and priority 100 `src_mac=...`.	Priority 200 row removed at release.

Terminal 2 shows: `[IDS ALERT] ... rule=multi_ip_spoof` and `[HARDWARE DEFENSE] Ingress Port 1 quarantined (Priority 200 DROP)`.

What to say: "A real computer has one IP address. When one MAC uses many, it is spoofing. Blocking one IP would not help, so the switch port is blocked."

4 SYN connection flood

Program sends	About 80 new connections per second to h2:5000, for up to 12 s. The program stops by itself when it is blocked.
Rule that fires	`conn_flood` at 20 new connections in 3 s.
Detected after	about 0.3 s (on the 20th connection)
Block installed	IP drop, priority 100, for 10.0.0.1.



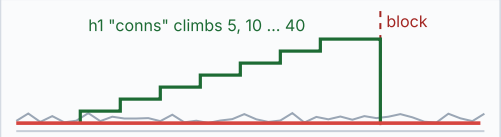
Panel part	While it runs	After the block
Graph	A short blip, often only one reading (tens to about a hundred packets/s). Because the block is so fast, the flood never gets big. Use the **log** button to see it.	Flat.
Live connections	`h1 → h2:5000` with up to about 20 conns, then the red BLOCKED row.	The ALLOWED row fades after about 10 s.
Traffic by service	The **5000 app** bar jumps up.	Back to normal.
Hosts table	h1 **conns** up to about 20, state **BLOCKED**. h2 **recv/s** rises for a moment.	h1 conns back to 0.
Activity feed	One or two lines `OPEN h1 → h2:app ×N`, then **ALERT conn_flood** ("20 new TCP connections in 0.x s"), then **BLOCK**.	**CLOSE** and later **RECOVER**.
Stat strip	**alerts** +1, **blocked** 1, **live connections** jumps by up to 20.	**dropped by IPS** shows a small count.

Terminal 2 shows: `[IDS ALERT] 10.0.0.1 | rule=conn_flood | 20 new TCP connections in 0.3s (limit 20)`

What to say: "Real clients open a few connections a second. Twenty in three seconds is a flood, and the controller sees every new connection, so it reacts at once."

5 Slow-and-low flows

Program sends	5 new connections per second for up to 30 s. Each connection is **held open**. This is too slow for conn_flood (5 × 3 s = 15, below 20).
Rule that fires	`concurrent_flows` at 40 open flows. Found by flow-counter polling every 3 s.
Detected after	about 8 to 11 s (40 connections take 8 s, then the next poll)
Block installed	IP drop, priority 100, for 10.0.0.1.



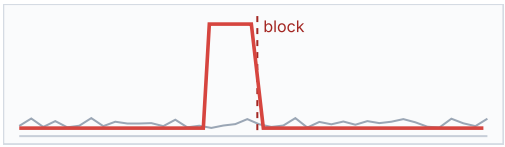
Panel part	While it runs	After the block
Graph	**No spike.** The packet rate stays low. This attack is dangerous for the table, not for the bandwidth.	Flat.
Live connections	`h1 → h2:5000` with the **conns** number climbing about 5 every second (5, 10, 15 ... 40 and more).	Row fades 10 s after the connections stop.
Stat strip	**live connections** climbs by the same amount.	Falls back.
Hosts table, h1	**conns** column climbs. State **ACTIVE** until the block, then **BLOCKED**.	conns back to 0.
Activity feed	Many `OPEN h1 → h2:app ×5` lines, then **ALERT concurrent_flows** ("N active flows (limit 40)"), **BLOCK**.	**CLOSE** lines, **RECOVER**.
Banner / Active blocks	Appear only at about 8 to 11 s. Wait for them.	Gone at release.

Terminal 2 shows: `[FLOW-STATS] 10.0.0.1 flows=N ...` every 3 s, then `[IDS ALERT] ... rule=concurrent_flows`.

What to say: "This attacker is slow, so a rate limit would miss it. Counting open connections in the switch catches it."

6 Volumetric flood

Program sends	4 connections to h2:5000 that send a very high number of tiny packets for 15 s.
Rule that fires	packet_rate at 3000 packets/s (found by polling every 3 s) **or** port_flood (see the box below).
Detected after	a few milliseconds (port_flood) or up to about 3 s (packet_rate)
Block installed	packet_rate: IP drop, priority 100. port_flood: port quarantine, priority 200 on port 1.



Panel part	While it runs	After the block
Graph	**The big peak.** The h1 line shoots up (thousands to hundreds of thousands of packets/s, depending on your laptop). The vertical axis rescales by itself. The orange dashed line (3000) is crossed. The peak is at its maximum in the first one or two readings after the flood starts.	The line falls to 0 on the next reading after the block.
Stat strip	**network packets/s** and **throughput** jump to the same size.	Fall back to normal.
Hosts table	h1 **sent/s** shows the huge number and **pkts sent** climbs very fast. h2 **recv/s** rises too. State **ACTIVE**, then **BLOCKED**.	sent/s 0. **dropped** shows the packets that arrived after the rule.
Live connections	`h1 → h2:5000` with 4 conns and a very high **pkts/s** (orange when above 1000).	The row fades; BLOCKED row stays.
Traffic by service	The **5000 app** bar fills the whole width.	Back to normal.
Banner, blocks, feed	Red banner, Active blocks card, **ALERT** and **BLOCK** lines.	**RECOVER** about 30 s after the last packet.

Terminal 2 shows: `[FLOW-STATS] 10.0.0.1 flows=4 pps=NNNNN` then `[IDS ALERT] ... rule=packet_rate | N packets/s (limit 3000)`.

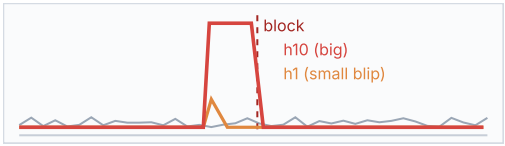
What to say: "A few connections can still carry a huge number of packets. The controller cannot see them, so it reads the switch counters every 3 seconds and blocks the source."

Option 6 can fire two different rules. Both are correct.

At the start of the flood, the first packets reach the controller before the fast-path rules are installed. If 35 or more arrive within 3 seconds, `port_flood` fires first (in milliseconds) and the **whole port** is quarantined (priority 200). If not, `packet_rate` fires at the next 3-second reading and only the IP is blocked (priority 100). You can tell which one happened from the banner: "(port N quarantined)" means port_flood. The graph peak is the same either way. To show the polling path on purpose, point to the `[FLOW-STATS]` lines in Terminal 2 and say that this is the second detection path.

c Coordinated attack (h1 + h10 together)

Program sends	h1 runs the SYN connection flood (option 4) and h10 runs the volumetric flood (option 6) at the same time for 14 s.
Rule that fires	h1: conn_flood. h10: packet_rate or port_flood.
Detected after	h1 about 0.3 s. h10 milliseconds to about 3 s.
Block installed	h1: IP drop for 10.0.0.1. h10: IP drop for 10.0.0.13, or port quarantine on port 13.



Panel part	While it runs	After the block
Graph	Two attackers: a small blip for h1 and the big peak for h10. All 11 normal lines stay low and keep moving.	h1 and h10 both fall to 0.
Red banner	Lists **both** attackers on one line (h1 and h10).	Gone at release.
Stat strip	**blocked** shows **2**, naming h1 and h10. **dropped by IPS** counts both.	Same.
Hosts table	h1 and h10 both **BLOCKED**. Every other row stays **UP** or **SERVING**.	Same.
Live connections	Two red BLOCKED rows on top. All the green ALLOWED rows of normal clients are still there and still changing.	Same.
Active blocks	Two cards, one per attacker, each with its own ttl.	Each disappears at its own release time.
Flow table (bottom)	Two DROP rows (priority 100 or 200).	Removed at release.

Terminal 2 shows: Two `[IDS ALERT]` lines (10.0.0.1 and 10.0.0.13) and two `[IDS RESPONSE] ... MITIGATED` lines.

What to say: "Two attackers at the same time, with different methods, are both stopped. The normal users on the dashboard did not notice."

The other keys, and the comparison of what stays the same

Key	What it does	What changes on the panel
7	Runs options 1 to 6 one after another from the selected attacker. Before each one it clears old blocks and afterwards it waits 4 s.	You see each pattern from this PDF in order, one block at a time. The whole run takes a few minutes (the flood takes 12 s, slow-and-low up to 30 s, volumetric 15 s).
a	Switches the attacker between h1 and h10. The menu shows the selected one.	Nothing changes until you launch an attack. After that, everything above happens for **h10 (10.0.0.13, port 13)** instead of h1.
8	Clears all blocks (deletes the DROP rules from the switch).	Within 1 to 2 s: the red banner disappears, BLOCKED rows go back to normal, Active blocks is empty, the feed shows **RECOVER**, and **dropped by IPS** returns to 0 because the rules (and their counters) are gone.
9	Prints the flow table in the terminal.	No change. It is the same data as the "OVS flow table" box at the bottom of the page.
0	Pings from the attacker (expect 100% loss) and from h3 (expect 0% loss).	Small change: the attacker's **dropped** number goes up by about 4 per ping run (pings are dropped by the block). h3's counters go up slightly.
s	Makes sure the servers and the background traffic are running.	If traffic was not running, the lines in the graph and the ALLOWED rows start to appear.
r	Prints the final report and writes `logs/deliverable2_results.csv`.	No change on the panel.
q	Quits the attack console.	No change. The controller, switch and background traffic keep running.

Things that always stay true during a demo

- The normal clients (h3 to h9) and the servers never get a BLOCKED label. They are in the protected list.
- The block is released 30 seconds after the attacker's last packet. If you press the key again too early, press **8** first.
- OPEN and CLOSE lines in the feed are normal background traffic. They are not attacks.
- If the header turns red ("controller: down" or "switch: down"), no attack will be detected. Run the clean-slate step and start the terminals again, in order.

> **A good order for a live demo**
> Show the calm dashboard (this is the normal-traffic graph). Press **6** for the big peak and the block. Press **8**. Press **3** to show a port being quarantined. Press **8**. Press **c** for two attackers at once. Point at the green ALLOWED rows that keep moving.